

17-214/514: Agentic Software Development

Evolution, Drift, and Decision Archaeology

Hammad Ahmad and Claire Le Goues • Wednesday, October 7, 2026 • Lecture

Administrivia

- A3 is due tonight.
 - A3 defense window will span before and after Fall break, but not during. There are no office hours during the break.
- A4 will be released tonight.
 - We will create your repos for you and send you an invite. Please accept the invite as soon as you get it, because it will expire in a week (during the break).

What did we talk about last time?

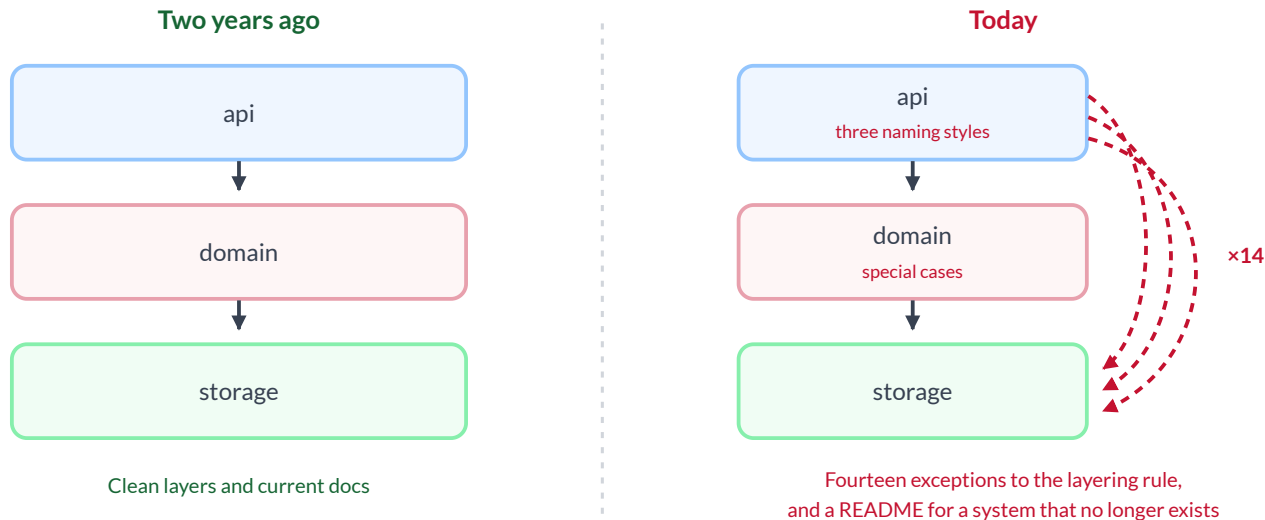
- An API is a promise made to strangers. The surface belongs to its callers, and everything behind it is yours.
- In code, types can make some wrong calls unwritable. Over a network, validation at the boundary is the nearest thing.
- LSP: a substitute must behave like what it replaces. ISP: no client should depend on methods it does not use.
- Hyrum's Law: with enough users, every observable behavior gets depended on. Evolve by adding, and deprecate with dates.
- A library is code you call, and a framework calls you. Adapter wraps what you do not own.
- Reuse or generate? It depends (on how much of the problem is domain-hard, and who maintains it).
- Today: what years of change do to a system, and how to read one you did not write.

Learning goals for today

- Explain why systems must evolve and how design decays under change
- Recognize architectural drift and the signals that reveal it
- Explain why agents can accelerate drift, and how preserved engineering memory counters it
- Read an unfamiliar codebase strategically (comprehension, architecture recovery, decision archaeology)
- Decide when to infer a lost decision from evidence and when to just ask a human

Two years of reasonable commits

Who made it this way?



Every commit in between was reviewed and approved.

Systems in use have to change

Software that people use accumulates demands (new features, platforms, scale, and rules).
Two of Lehman's laws:

I. Continuing change

A system that is used **must** be continually adapted, or it becomes progressively less useful.

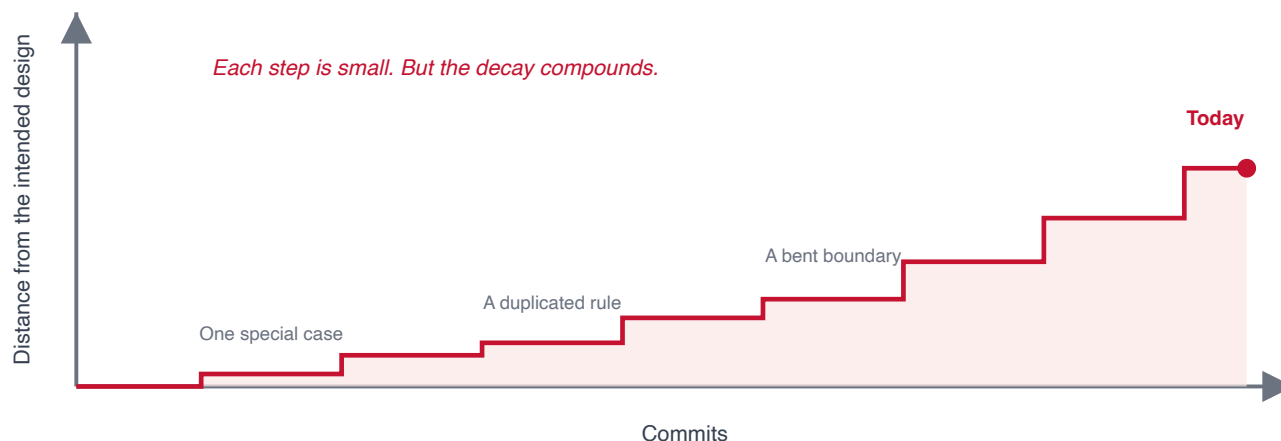
II. Increasing complexity

As a system evolves, its complexity increases **unless work is done to reduce it.**

For example, our booking system keeps gaining features (cancellations, a waitlist, pricing rules).

Design decay

- A design rarely breaks in one change. Each change may bend a boundary a little, duplicate a little knowledge, or add one special case, etc.
- Refactoring is one way to do the work Lehman's second law calls for (reducing complexity as the system changes).



Architectural drift

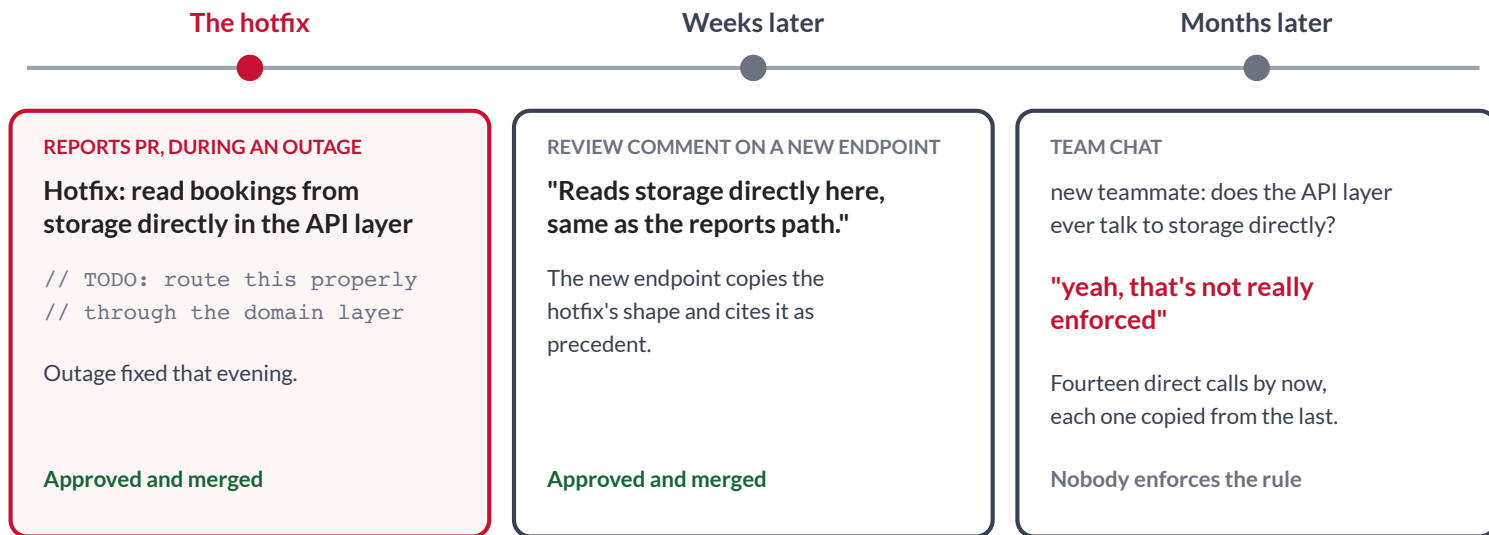
- The design says one thing, but the implementation, over time, does another.
- The design in everyone's head can stay clean while the code changes.
 - Our reasoning now describes an older version of the system.

13 years ago	Fixes ui/views/corewm/DEPS	9	]
		10	
		11	# TODO: temporary, don't add more.
		12	specific_include_rules = {
		13	"tooltip_controller.cc": [
		14	"+ui/views/widget/tooltip_manager
		15	],
		16	
		17	"tooltip_aura.cc": [
3 years ago	cros_next: add ash style toolti...	18	"+ui/views/widget/widget.h",
2 years ago	[views-ax] Migrating Role to vie...	19	"+ui/views/accessibility/view_acc
3 years ago	cros_next: add ash style toolti...	20	],
		21	
		22	"tooltip_view_aura.cc": [
2 years ago	[views-ax] Migrating Role to vie...	23	"+ui/views/accessibility/view_acc

github.com/chromium/chromium/blame/main/ui/views/corewm/DEPS

A drift autopsy

Our layering rule: **the API layer never touches storage directly.**

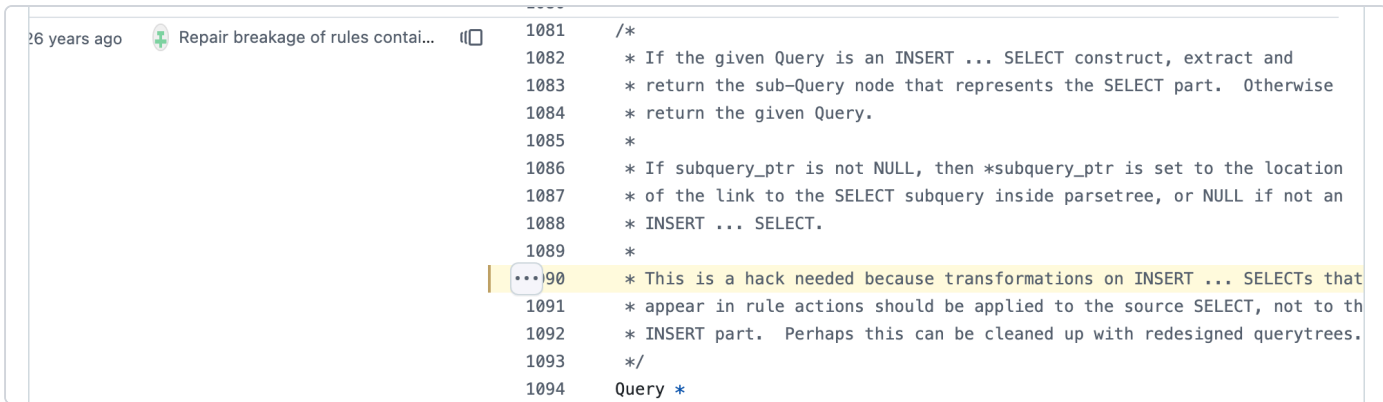


The rule was never changed. Each exception was reasonable, but none was recorded.

Signals of drift

Not an exhaustive list, but each one can mean the intent and the system have diverged:

- Old TODOs and dated "hacks".
- Dead feature flags that aren't deleted, and exception lists that keep growing.
- The one module everyone routes around, and the doc that describes the system as it was a number of changes ago.



```
26 years ago  Repair breakage of rules contain... 1081  /*
1082  * If the given Query is an INSERT ... SELECT construct, extract and
1083  * return the sub-Query node that represents the SELECT part. Otherwise
1084  * return the given Query.
1085  *
1086  * If subquery_ptr is not NULL, then *subquery_ptr is set to the location
1087  * of the link to the SELECT subquery inside parsetree, or NULL if not an
1088  * INSERT ... SELECT.
1089  *
1090  * This is a hack needed because transformations on INSERT ... SELECTs that
1091  * appear in rule actions should be applied to the source SELECT, not to th
1092  * INSERT part. Perhaps this can be cleaned up with redesigned querytrees.
1093  */
1094  Query *
```

github.com/postgres/postgres/blame/master/src/backend/rewrite/rewriteManip.c#L1090

Lost assumptions

- "The store assumes bookings arrive sorted." True since version 1, but not written down or tested.
- Two years later, a new import path inserts unsorted. Nothing crashes, but some queries return wrong answers.
- The assumption was forgotten, then broken without anyone noticing.

"mem" and "object" Allocators are No Longer Protected by the GIL #105766

✓ Closed



ericsnowcurrently

opened on Jun 13, 2023

Last edited by bedevere-bot

Member



Once we moved to per-interpreter GIL, the promises in [the docs](#) no longer hold:

...where the allocation must be performed with the GIL held.

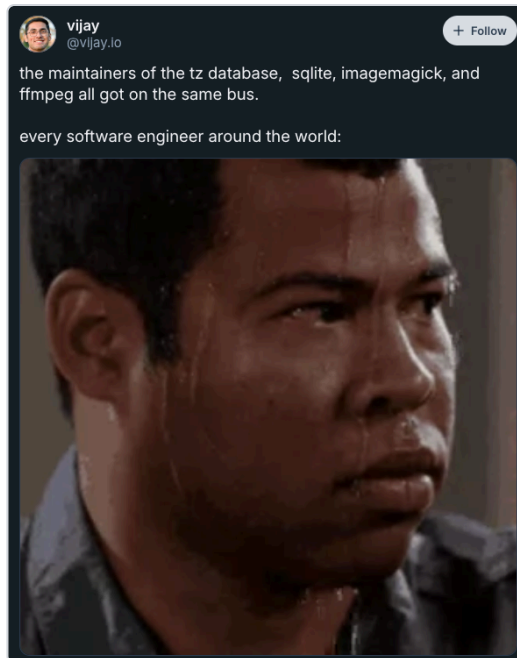


It's still fine for pymalloc, but any custom, non-wrapping "mem"/"object" allocators would need to be updated to be thread-safe or per-interpreter.

github.com/python/cpython/issues/105766

The constraint in one engineer's head

- Many teams have a constraint that lives in one engineer's head.
- It works until they graduate, switch teams, or win the lottery.
- (Yep, that's the bus factor, or the lottery factor if you prefer the nicer version.)



programmerhumor.io

Whodunit?

Usually not one person per se.

- The sum of reasonable changes did.
- Nobody has to act carelessly for drift to happen.
- Generally, "carefully hire careful people" does not prevent drift on its own.

Now add an agent

- Everything so far happened at human speed, with people who half-remember last month's intent.
- An agent generates fast. It follows the prompt and the patterns in the code it reads, exceptions included.
 - In a fresh session, it knows the layering rule only if the rule is in its context.
- Many slightly off generated changes can add up in weeks instead of months.
- Cheaper changes mean more changes, and each one can add complexity (Lehman's second law).

Conventions can change over time

- Over time, generations may follow a slightly different convention (`getUser` , `fetchRoom` , `loadBooking`).
- Naming style may well tell you when code was generated ("ah, this is from the Opus 4.7 era").

When you're working on a file that
was last updated six years ago



programmerhumor.io/programming-memes/the-archaeological-expedition-into-legacy-code-ebyl

Does documentation still rot?

- Agents update the docs they can see, often eagerly. The docs they are not pointed at can still rot.
- For what the system does today, trust the code first, then the tests, then the README, and the wiki least.
- A README describes the code. If an agent updates it to match drifted code, the README now describes the drift as if it were intended.
- The contract and the decision records say what the system should do. Checking the code against them is how drift shows up.

Recording decisions

What if the hotfix had been written down?

Recall deliberate debt and accidental mess. The autopsy's hotfix was a deliberate shortcut, but it was not written down.

What the hotfix left

```
// TODO: route this properly through  
the domain layer
```

No owner and no end date. The next endpoint copied it as precedent.

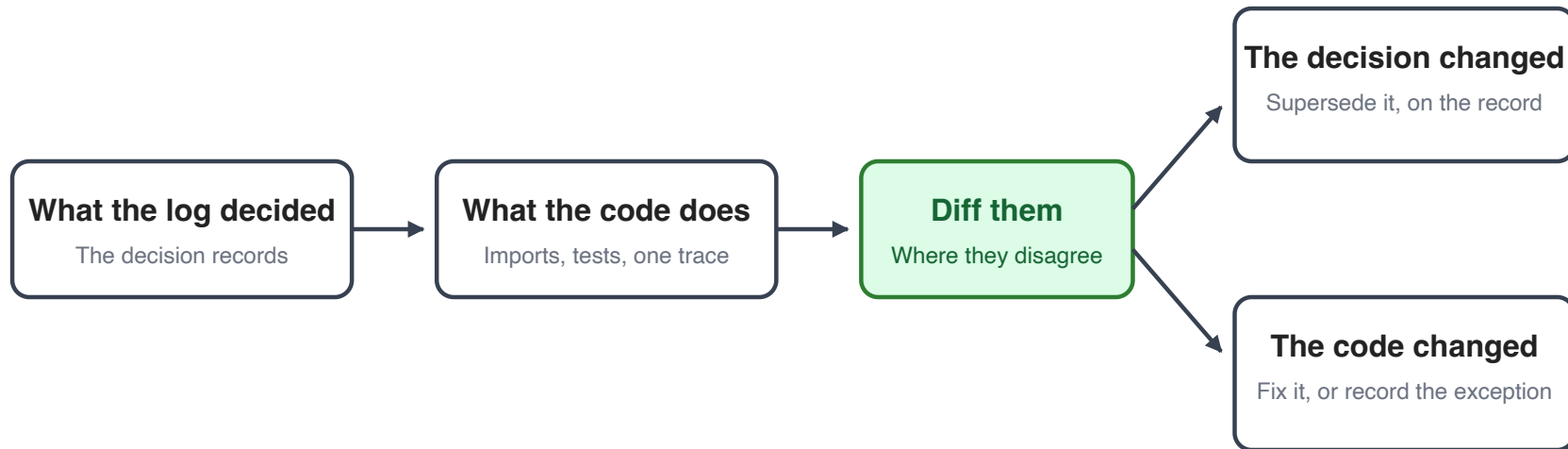
ADR 016: Temporary exception to ADR 003

The reports path reads storage directly, to fix the outage. Revisit before any other endpoint does the same.

Status: accepted, temporary.

Same shortcut, but a reviewer can find the rule it breaks and when it was meant to end.

Decision records can detect drift



This audit is a **drift check**.

It only covers what was recorded (usually the few decisions that are hard to undo). Expect to run one on your own project later this semester.

The same rule as a test

The autopsy's rule:

```
@ArchTest
static final ArchRule apiNeverTouchesStorage =
    noClasses().that().resideInAPackage("..api..")
        .should().dependOnClassesThat().resideInAPackage("..storage..");
```

- Recall architecture tests: the oracle is a structural decision you made.
- The autopsy's hotfix fails the build the day it is written. Either the exception gets recorded or the code gets routed properly.
- Run it where the agent sees it fail (a hook, or the local test run). If it fails only in CI, the agent finds out late.

An import rule in Kubernetes

Kubernetes made its scheduler's current in-repo imports an allowlist. A new one fails the build.

```
1  rules:
2    # Allow only specific k8s.io/kubernetes dependencies currently used
3    - selectorRegexp: ^k8s[.]io/kubernetes/pkg/api/v1/pod$
4      allowedPrefixes: [ "" ]
5    - selectorRegexp: ^k8s[.]io/kubernetes/pkg/apis/core/v1/helper$
6      allowedPrefixes: [ "" ]
7    - selectorRegexp: ^k8s[.]io/kubernetes/pkg/apis/scheduling$
8      allowedPrefixes: [ "" ]
9    - selectorRegexp: ^k8s[.]io/kubernetes/pkg/controller/volume/persistentvolume/testing$
10     allowedPrefixes: [ "" ]
11    - selectorRegexp: ^k8s[.]io/kubernetes/pkg/features$
12     allowedPrefixes: [ "" ]
13    - selectorRegexp: ^k8s[.]io/kubernetes/test/utils/ktesting($|/)
14     allowedPrefixes: [ "" ]
15
16    # Allow internal pkg/scheduler packages and subpackages
17    - selectorRegexp: ^k8s[.]io/kubernetes/pkg/scheduler($|/)
18     allowedPrefixes: [ "" ]
19
20    # Forbid any other k8s.io/kubernetes imports
21    - selectorRegexp: ^k8s[.]io/kubernetes
22     forbiddenPrefixes: [ "" ]
```

github.com/kubernetes/kubernetes/pull/141415

Let's triage a drift check

Our booking system keeps a decision log. Three of its records:

- **ADR 003.** The API layer reaches storage only through `BookingService`.
- **ADR 014.** The waitlist is first-come, first-served.
- **ADR 015.** Cancelling less than 24 hours before the start costs a fee. Staff bookings are exempt.

Two things happened recently. The product owner approved waitlist priority for premium members, and `BookingService` gained `bookAll()` for bulk bookings.

- **ADR 003.** The API layer reaches storage only through `BookingService`.
- **ADR 014.** The waitlist is first-come, first-served.
- **ADR 015.** Cancelling less than 24 hours ahead costs a fee. Staff bookings are exempt.
- **Recently.** Premium waitlist priority was approved. `BookingService.bookAll()` exists now.

An agent's drift-check report:

```
1 api/ReportsController.java:42
  Reads BookingStore directly. Violates ADR 003.
2 domain/Waitlist.java:88
  Orders premium members first, then by request time. Violates ADR 014.
3 domain/CancellationPolicy.java:31
  Waives the late fee when user.isStaff(). Violates ADR 015.
4 api/BookingController.java:57
  Bulk endpoint writes BookingStore directly. Violates ADR 003.
  Marked "TODO: temporary, until BookingService supports bulk".
```



Canvas → Quizzes
→ Lecture 12
In-Class Quiz

- For each finding: fix the code, supersede the record, or record an exception?
- Is any finding wrong?
- Would your answer to #1 change if a second report were coming next week?

The same request in two codebases

The activity's feature: **waitlist priority for premium members.**

With the log

Read the three records. Route the feature through `BookingService` (ADR 003), and supersede the waitlist record (ADR 014) with the premium exception.

Without the log

Work out why the waitlist orders the way it does. You may well guess wrong (e.g., it looked accidental, but it was a deliberate fairness rule). Ship, then prepare to deal with the support tickets.

Recall the course thesis

- Generation made code cheap. Understanding is still expensive.
- A written record means the next reader looks the decision up instead of working it out again.
- The cheaper code gets, the more the memory around it is worth.
- But, remember that a few good records are better than a pile of boilerplate ones. Focus on the important decisions.
 - Agents can write the pile quickly. Every doc in it is one more thing that can go stale.

Instruction files rot too

- A stale instruction file can mislead an agent, which may follow it without noticing.
- Instruction files are decisions the agent acts on, so they need the same discipline as the log. Update them in the same commit as the change that invalidates them.

The original CLAUDE.md

```
Use the legacy date library  
(moment) for all date math.
```

The project has since moved to date-fns

The agent, today

```
Following the project  
conventions, I'll use moment...
```

Obeys a stale rule

Reading unfamiliar code

Now you inherit the codebase

- So far you were the one keeping the memory. Now suppose you arrive and there is none (or very little, or outdated memory).
 - No log, a README from 2019, and three naming conventions.
- This is very common, by the way. You'll have to navigate codebases like this throughout your careers.

You won't understand the whole system

- Most code you will change was written by someone or something else.
- The goal is a working model of the part you will change (its pieces, and how they connect).
- You build the model, test it, and fix it where it was wrong.
- So how do you read a system you have never seen?

How do experienced readers read code?

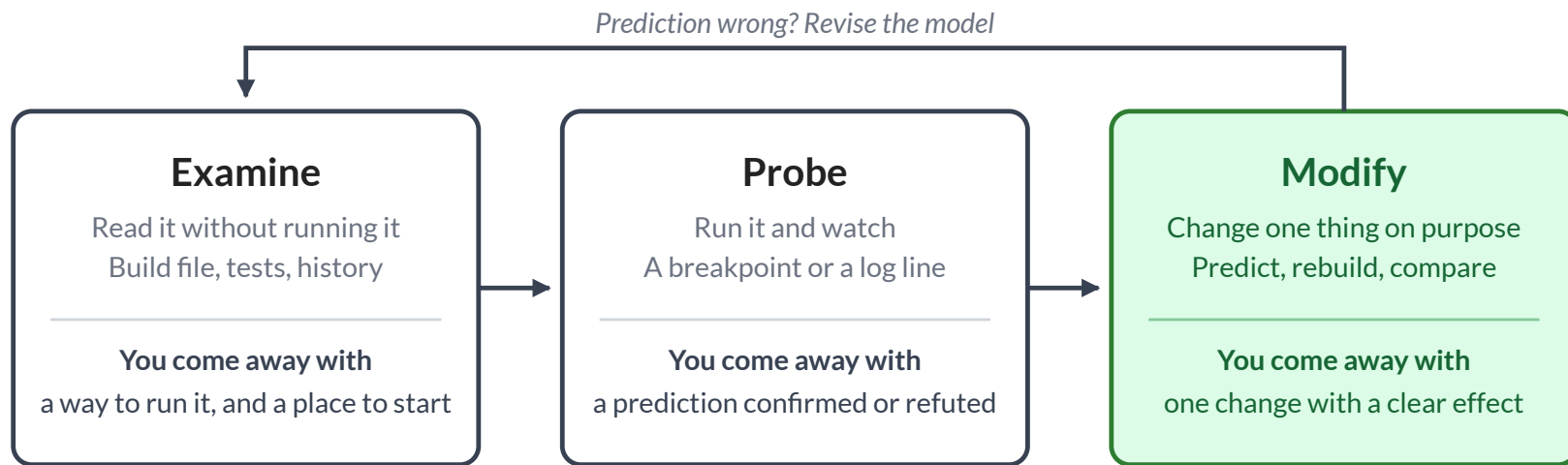
A less experienced reader tends to

- Read line by line, from the top.
- Re-read the same code.
- Try things until something works.
- Check only the happy path.

A more experienced reader tends to

- Start from the overall structure.
- Recognize patterns (file layout, architecture, names).
- Form a hypothesis, then check it.
- Check what calls the code you will change, and what it calls.

Examine, probe, modify



The same reading order at a larger scale

Recall: reading a generated change

- Glance at the file list.
- Let the agent search, and report.
- Spot-check the report.
- Read closely where the risk is.

Reading a whole repository

- Glance at the build file and the directory shape.
- Let the agent trace one request, and report.
- Spot-check the trace.
- Read closely where your change goes.

Exploring Jenkins

"What are the parts?"

(Note: the first question is not "how does it work?")

```
$ ls
bom cli core coverage docs
pom.xml test war websocket ...

$ grep '<module>' pom.xml
  <module>bom</module>
  <module>websocket/spi</module>
  <module>websocket/jetty12-ee9</module>
  <module>core</module>
  <module>war</module>
  <module>test</module>
  <module>cli</module>
  <module>coverage</module>
```

- The build file lists eight Maven modules.
- `core` (1,254 Java files) holds the `Queue` class we trace next, so that's where we will focus.
- `test` is its own module, so functional tests live apart from unit tests.
- No code read yet, but we already have the boxes.

One request end to end

"Trace what happens when a job is scheduled until it runs on an executor. Cite file and line."

1 Queue.java:582	schedule2()	public entry point
2 Queue.java:636	new WaitingItem	enters waiting
3 Queue.java:1594	maintain()	moves items on
4 Queue.java:1702	makeBuildable()	quiet period over
5 Queue.java:1789	loadBalancer.map()	picks an executor
6 Queue.java:253	executor.start()	hands over the work
7 Executor.java:810	start(WorkUnit)	starts the thread
8 Executor.java:456	queue.execute()	the build begins

- Eight steps/claims (the agent says the code does something at that file and line).
- We spot-check the steps that later steps depend on, or that a change would touch.
- Step 7 is one of them (step 8 runs on the thread it starts), so we open `Executor`.

```
// Executor.java:93
public class Executor extends Thread
    implements ModelObject, IExecutor {
```

- Yep. Each executor is its own thread. Cool.

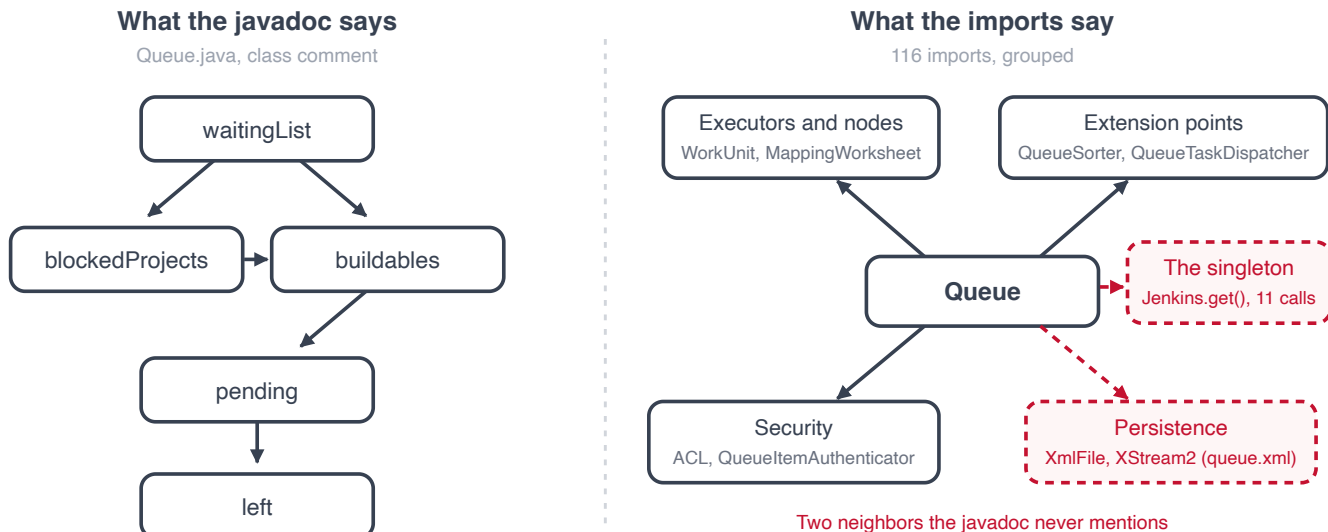
The tests as a table of contents

```
$ cd test/src/test/java/hudson/model
$ ls | grep -i queue
JobQueueTest.java
LabelLoadStatisticsQueueLengthTest.java
QueueCrashTest.java
QueueRestartTest.java
QueueTest.java
queue

$ ls -l queue/
BuildKeepsRunningWhenFaultySubTasksTest.java
LoadPredictorTest.java
MaintainCanTakeStrengtheningTest.java
QueueTaskDispatcherTest.java
WideExecutionTest.java
```

- Ten test classes for one 3,267-line class, and the names say what the queue promises.
- From the names alone, `QueueCrashTest` and `QueueRestartTest` suggest the queue is saved to disk and survives a restart.
- Three of them carry disabled tests, each with a reason left behind ("flakes in CI", "Pending JENKINS-68319").

Draw what the code implies



The javadoc shows the stages an item moves through. The imports add saving to disk and a global `Jenkins` object. (Reminder: the docs can be stale.)

Have your agent draw the right half from the imports, then spot-check the red boxes.

Probe: run it and watch

Prediction: triggering a build calls `Queue.schedule2()` (step 1 of the trace). To check, set a breakpoint there with `jdb` (the JDK's command-line debugger) and trigger a build.

```
$ jdb -attach 5005          # connect to the running Jenkins
> stop in hudson.model.Queue.schedule2(hudson.model.Queue$Task,int,java.util.List)
    (trigger a build)
Breakpoint hit: "thread=Handling POST /jenkins/job/demo/build ...",
    hudson.model.Queue.schedule2(), line=584
> where                    # who called it?
[1] hudson.model.Queue.schedule2 (Queue.java:584)
[2] hudson.model.Queue.schedule2 (Queue.java:713)
[3] jenkins.model.ParameterizedJobMixIn.doBuild (ParameterizedJobMixIn.java:230)
```

- It held. The breakpoint was hit on the thread handling the HTTP request.
- The stack also shows two callers the trace left out (`doBuild` and a second `schedule2()` overload).
- An agent ran this probe. We predict and check the results.

Modify: change one thing on purpose

Prediction: the quiet period (step 4 of the trace) is what holds a new build back. With a default of 0, builds should start at once.

- It held, but the shipped wait was 7 to 9 seconds instead of 5.
 - The queue moves items forward on a 5-second timer (`Queue.java:3039`). That timer was missing from our model.
- Revert the change afterward, and keep the result in your notes.

```
// core/src/main/java/jenkins/model/Jenkins.java:2341
public int getQuietPeriod() {
-   return quietPeriod != null ? quietPeriod : 5;
+   return quietPeriod != null ? quietPeriod : 0;
}
```

Default	Trigger to build start, 3 runs
5 s (as shipped)	7.4 s, 8.8 s, 8.9 s
0 s (changed)	14 ms, 5 ms, 3 ms

An agent made the edit and timed three runs.
We predict and check the results.

What to write down while you read

- The up-to-date drawing(s).
- The one request you traced, with file and line per step.
- Each probe and change in one line (what you did, the code that ran, what you saw).
- Your open "why" questions (for example, why does `Executor` extend `Thread` ?).
- Related: A4 asks for a citation behind every claim. Keep those as you go.
 - Friday's lab is the small version of touching code like this (a characterization test first, then one refactor).

Reading a new (to you) codebase with an agent

- Earlier, you reviewed changes the agent wrote, against what you asked for. Here it explains code neither of you wrote.
- Fast: "summarize this module", "trace this request", "map what imports what".
- Risky: a plausible summary of code it sampled rather than read.
- Ask it to separate what it checked from what it assumed.
- Without your agent, you should be able to explain the part you will change, at a high level.
 - For the rest, knowing roughly what each part does is usually enough.
- For A4: you cannot read all 200K+ lines, and we don't expect you to.
 - As a reminder, read closely where your change goes and where the risk is, skim the structure everywhere else, and skip the corners you won't touch.

Decision archaeology

Digging for decisions

- The code says what it does. Archaeology recovers why, from evidence.
- The evidence: the project's own docs, git history and blame, commit messages, issue links, test names, config defaults, and code that is odd in a specific way.
- Some decisions are recorded in detail, many only partly, and a few not at all.
 - Records change too, so read the version in force at the date you are judging.
- Chesterton's fence: do not remove the odd thing until you know why it is there.

PM: The last dev wrote a really in-depth handover document

The handover document:



programmerhumor.io/devops-memes/handover-idn3

"Why don't reads take the queue's lock?"

Exhibit: `Queue.java:218` keeps a `volatile Snapshot` (all threads see updates immediately), and reads use it instead of locking.

The dig:

The history

```
$ git log --reverse \  
    -S'updateSnapshot'  
92147c3597 2015-03-23  
[FIXED JENKINS-27565]  
Refactor the Queue and Nodes  
to use a consistent locking  
strategy
```

The commit body

```
- Remove any requirement  
from Jenkins Core to  
lock on the Queue when  
rendering the Jenkins UI  
[FIXED-JENKINS-27566]
```

The older record

```
getApproximateItemsQuickly()  
    "primarily added to make  
    UI threads run faster"  
@since 1.483  
@deprecated ...  
    As of 1.607
```

Then judge the code against each record (matches, drifted, or moved past). Here it matches the 2015 commit, and the older javadoc was deprecated to match.

"Why does Executor extend Thread?"

Exhibit: `Executor.java:93`, one thread per executor slot.

The dig:

The first commit

```
$ git log --diff-filter=A \
  -- '*/Executor.java'
8a0dc230f4 2006-11-05
kohsuke
"initial commit for
a new layout."
```

The complaint, nine years later

```
Queue.java:1614
// (All this dancing is
//  a result of Executor
//  extending Thread)
```

That commit's message

```
6f343dc7c2 2015-06-15
[FIXED JENKINS-28840]
Deadlock between
Queue.maintain and
Executor.interrupt
```

The judgment: does each piece show a reason or a consequence? Here, only consequences. Your ADR marks the reason unknown, names the alternative (tasks on a pool), and cites the consequences.

Sometimes it is better to stop digging and ask instead

- Inference can be wrong, but people *may* remember the big decisions. If the engineer who built it is in the same office or a quick Slack message away, ask them.
- Then write down what they tell you, so the next reader digs less.



programmerhumor.io/programming-memes/ancient-scriptures-k5tj

Judging a recovered decision

- A recovered decision can be judged again. Was it right then, and is it still right now?
- A thread per executor was ordinary in 2006. Today `maintain()` needs a workaround for it to avoid a deadlock.
- Decisions often age this way as requirements change.
- The problems you find are where your candidate refactor comes from.

Summary

- Used systems must change (Lehman), and unmanaged change makes them more complex. Drift can come from reasonable exceptions alone.
- Agents can accelerate drift. They generate fast, copy the patterns they see (exceptions included), and over time generations may follow slightly different conventions.
- Recorded decisions can slow drift. The decision log can detect it (for what was recorded), superseding keeps evolution on the record, an architecture test enforces a decision, and instruction files need the same discipline.
- When you inherit a system, examine it (the build file, the tests, one traced request, what the imports imply), probe it by running it, change one thing on purpose, and recover decisions from the history before you judge them.
- Chesterton's fence stands until you know why it is there. When a person knows, ask them, then write it down.

What's next

W1 A1 out	W2	W3 A1 due	W4	W5 A2 due · M1	W6	W7 A3 due	W8	W9 A4 due	W10 M2	W11	W12 A5 due	W13	W14 A6 due
--------------	----	--------------	----	-------------------	----	--------------	----	--------------	-----------	-----	---------------	-----	---------------

- **Friday, Lab 7:** Direct a refactor with a characterization test first, then critique the patterns in the code.
- **Fall Break:** No lectures next week and no lab next Friday. No office hours either. Rest is a maintenance activity too. :-)
- **When we return:** Agent architectures.
- **A4 checkpoint, Wed Oct 21:** `ARCHAEOLOGY.md`. The proposal, tests, and refactor are due Oct 28.